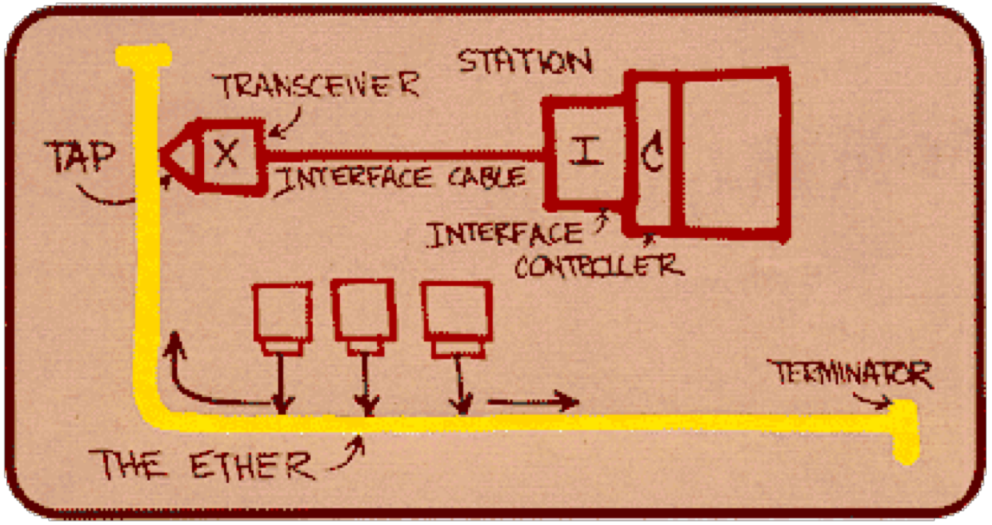
Ethernet

"dominant" wired LAN technology:

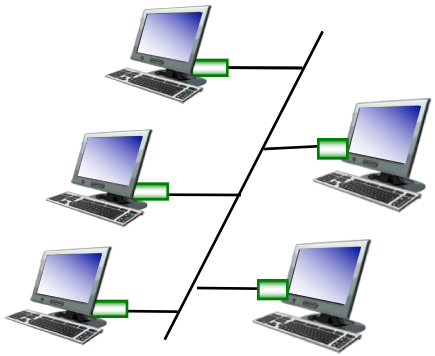
- single chip, multiple speeds
- first widely used LAN technology
- simpler, cheap
- kept up with speed race: 10 Mbps – 10 Gbps



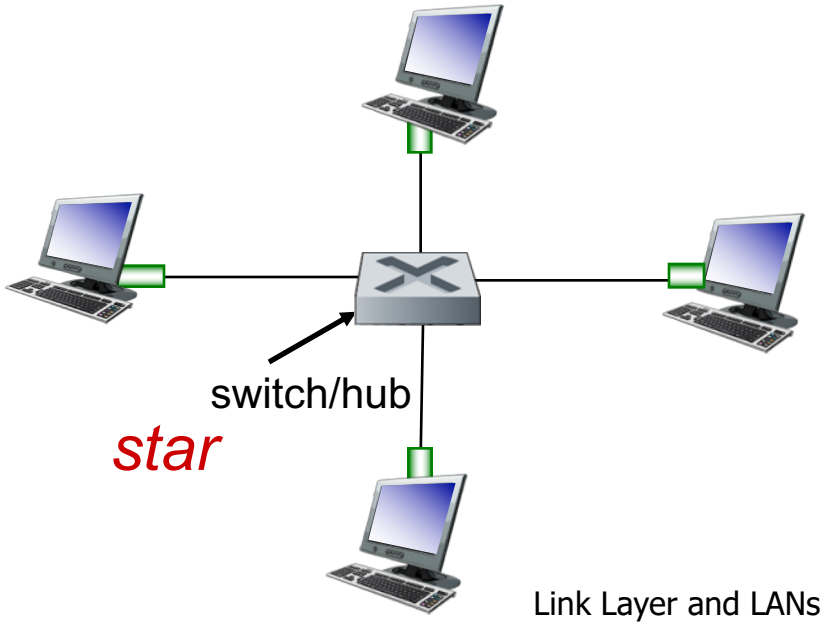
Metcalfe's Ethernet sketch

Ethernet: physical topology

- *bus:* popular through mid 90s
  - all nodes in same collision domain (can collide with each other)
- *star:* prevails today
  - active *switch/hub* in center
  - each "spoke" runs a (separate) Ethernet protocol
  - switch: nodes do not collide with each other, buffers used
  - hub: all spokes broadcast their transmissions, no buffering



bus: coaxial cable



Ethernet frame structure

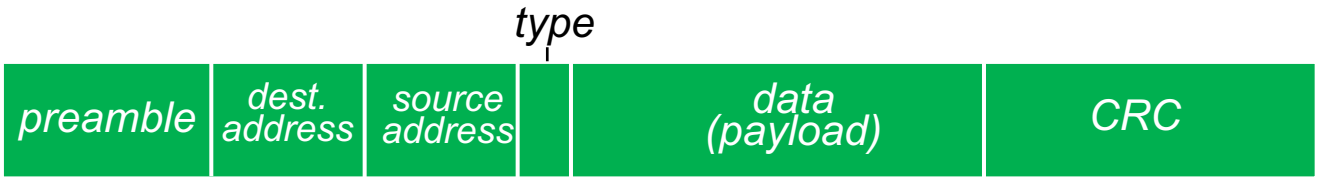
sending adapter encapsulates IP datagram (or other network layer protocol packet) in Ethernet frame

preamble:

- 7 bytes with pattern 10101010 followed by one byte with pattern 10101011
- used to synchronize receiver, sender clock rates

Ethernet frame structure (more)

- *addresses:* 6 byte (octet) source and destination addresses
  - if adapter receives frame with matching destination address, or with broadcast address, it passes data in frame to higher layer protocol
  - otherwise, adapter discards frame
- *type:* indicates which higher layer protocol carried in frame
- *CRC:* cyclic redundancy check bits used by receiver to detect error
  - error detected: frame is dropped, no recovery

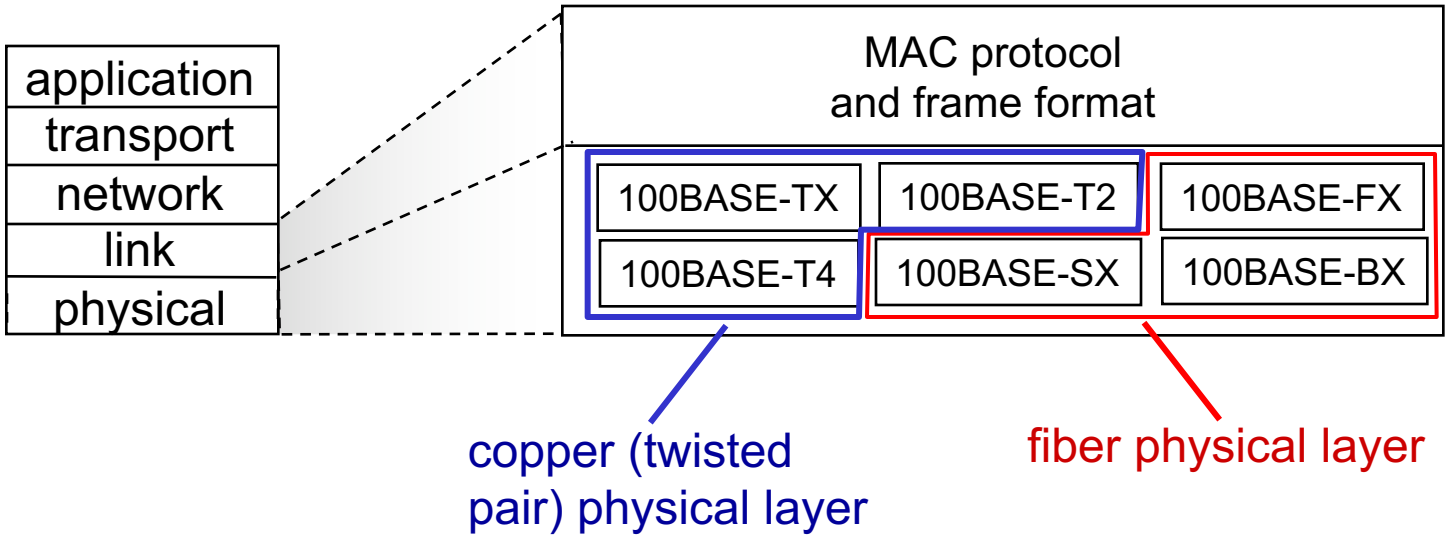


Ethernet: unreliable, connectionless

- *connectionless:* no handshaking between sending and receiving NICs
- *unreliable:* receiving NIC doesn't send acks or nacks to sending NIC
  - data in dropped frames not recovered
  - relies on higher layer reliable data transport (e.g., TCP), otherwise dropped data lost
- Ethernet's MAC protocol: unslotted *CSMA/CD with binary exponential backoff*

802.3 Ethernet standards: link & physical layers

- *many* different Ethernet standards
  - common MAC protocol and frame format
  - different speeds: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps, 10 Gbps, 40 Gbps
  - different physical layer media: fiber, cable



Link layer, LANs: outline

6.1 introduction, services

6.2 error detection, correction

6.3 multiple access protocols

6.4 LANs

- Ethernet
- addressing, ARP

Device interface addresses

- interface address (referred to as: MAC/LAN/Ethernet/physical/hardware address):
  - function: *used 'locally" to get data frame from one device interface to another device interface on **same** physical layer network – Ethernet, WiFi,…..,*
  - address bits burned in NIC ROM, also sometimes software settable
  - e.g.: Ethernet – 48bits (6octets): 1a-2f-bb-76-09-ad

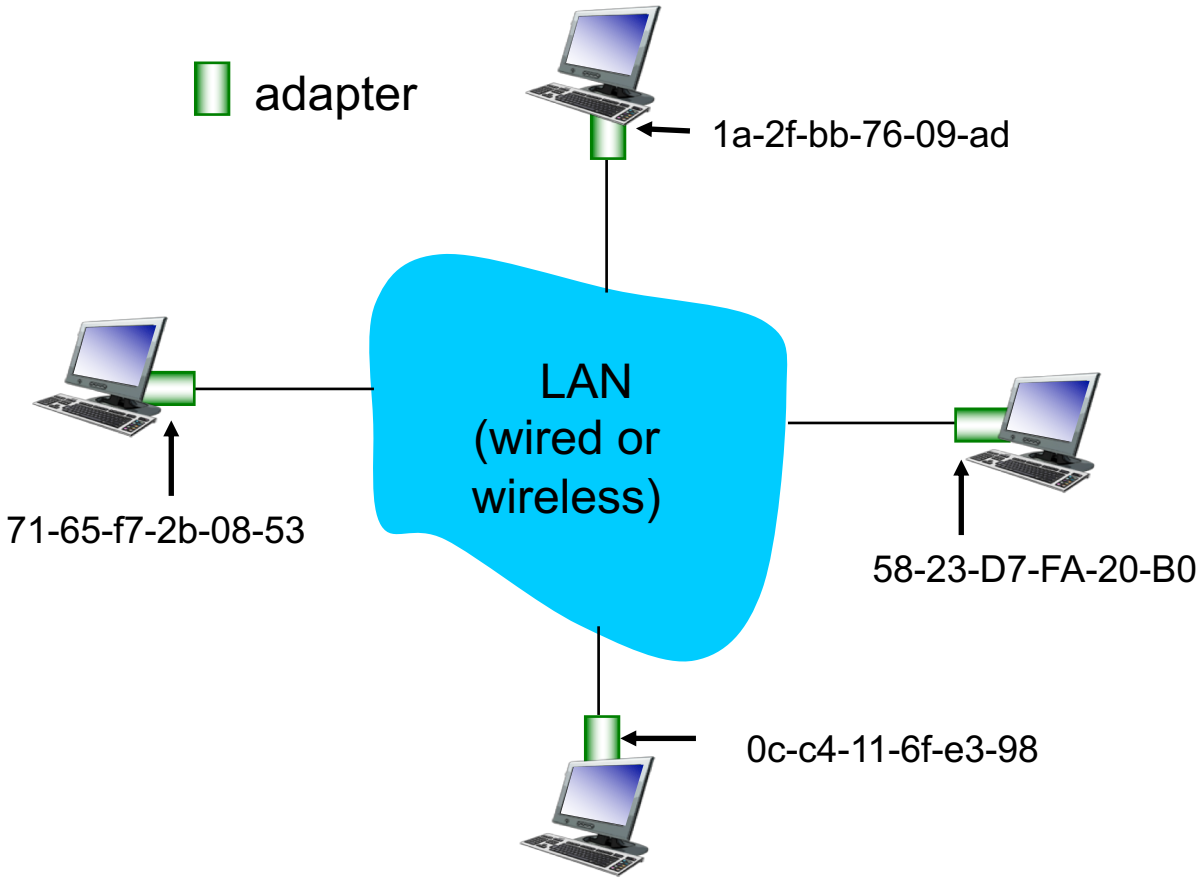
hexadecimal (base 16) notation
(each "numeral" represents 4 bits – 12x4=48)

1=0001, 9=1001, 10=a=1010
11=b=1011, 14=e=1110, 15=f=1111

  - ff:ff:ff:ff:ff:ff – all "1"s -> broadcast address on local network

MAC addresses

- each adapter on LAN has unique *LAN* MAC/hardware address
- devices can have more than one interface/adapter –> one MAC address per adapter (Ethernet, WiFi, cellular)



MAC addresses (more)

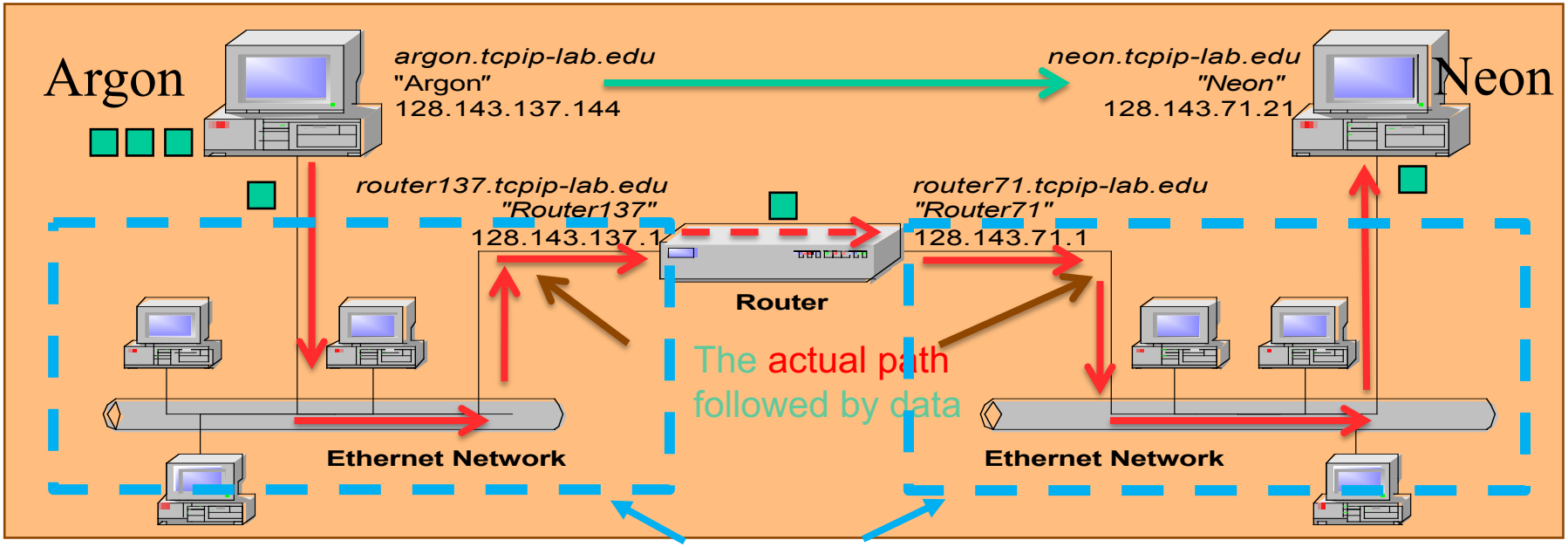
- MAC address allocation administered by IEEE
- manufacturer buys portion of MAC address space (to assure uniqueness)
- analogy:
  - MAC address: like Social Security Number (SSN)
  - Internet address: like postal address
- MAC -> flat address ➜ portability
  - can move LAN card from one LAN to another
  - people move - their SSN goes along, does not change
- Internet -> hierarchical address (network-host) ➜ *not* portable
  - address based on which network the device is currently connected to, e.g., UCI, Cox OC, Sprint, Starbucks
  - people move - their postal address changes

How to determine device MAC addresses

Question: how to determine a device's MAC address, if all you have is its internet (i.e, IP) address?

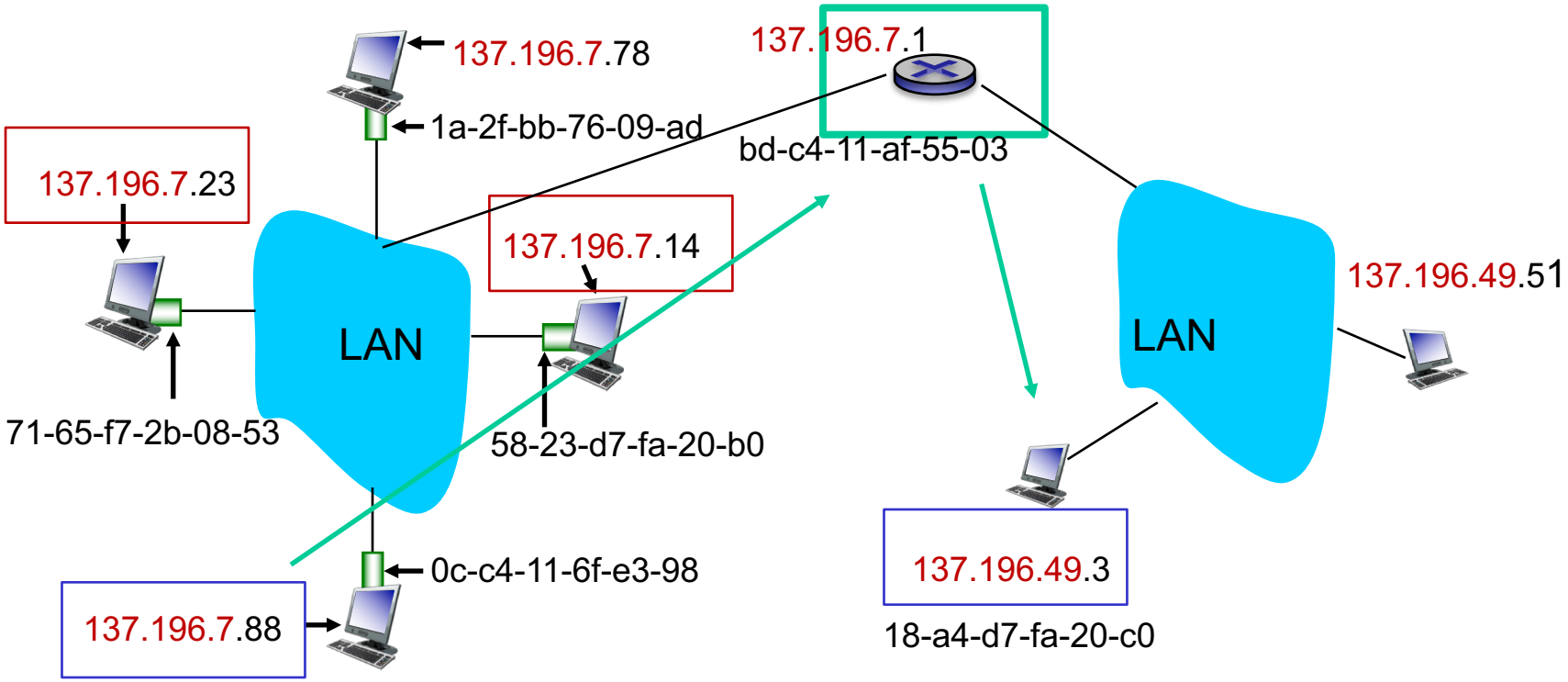
Answer: ask for it! Address Resolution Protocol - ARP

Note: only needed for local/adjacent devices (recall IP hop-by-hop transmission)



Local/Adjacent devices

All devices are on same "subnet"



ARP protocol: same local net

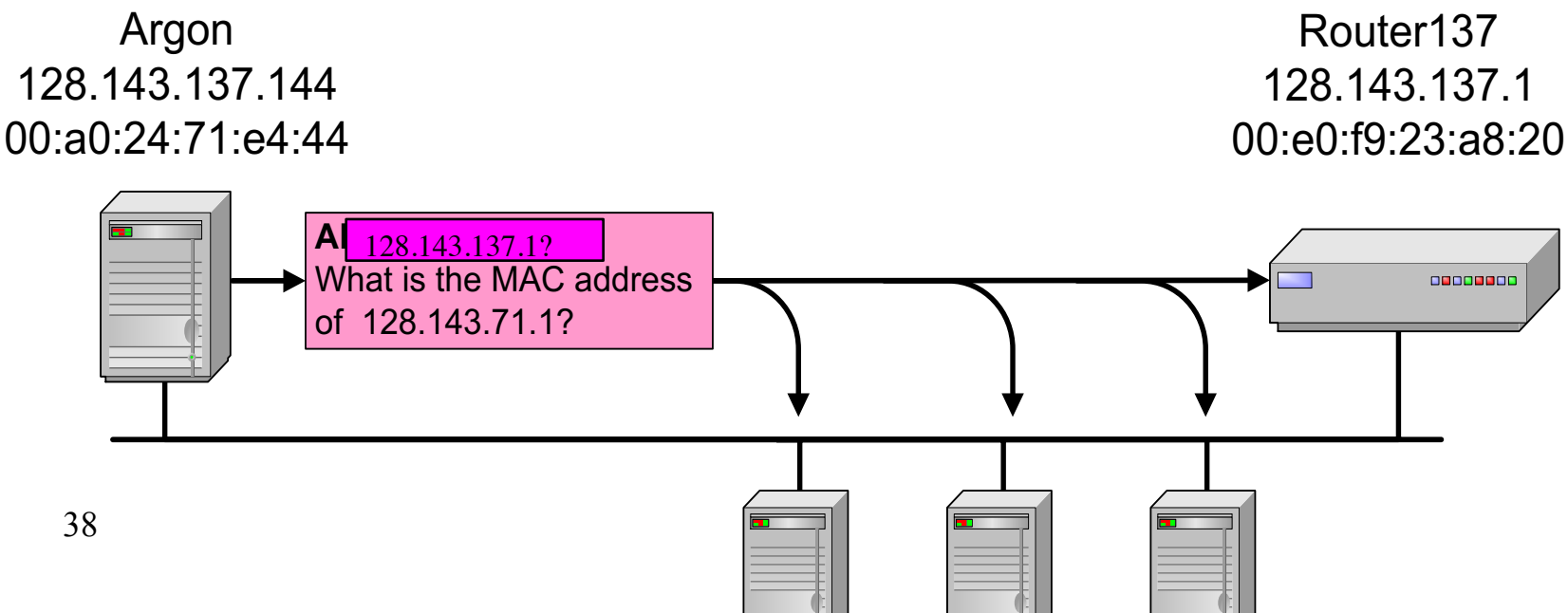
- A wants to send datagram to B:
  - B has to be on same local (physical) network (more later…)
  - B's IP address known to A
  - B's MAC address not known to A
- A broadcasts on **local** network ARP request packet, containing B's IP address in data field
  - destination MAC address of data link frame = ff:ff:ff:ff:ff:ff
- all nodes on local network receive ARP request and read
- B recognizes its IP address in request
- B then sends an ARP reply **to** A with its (B's) MAC address in the data field
  - destination MAC address of data link frame = A's MAC address (B extracted A's MAC address from received APR request).

Address Translation with ARP

ARP Request:

Argon broadcasts on data link layer (address: ff:ff:ff:ff:ff:ff) an ARP request to all stations on the network:

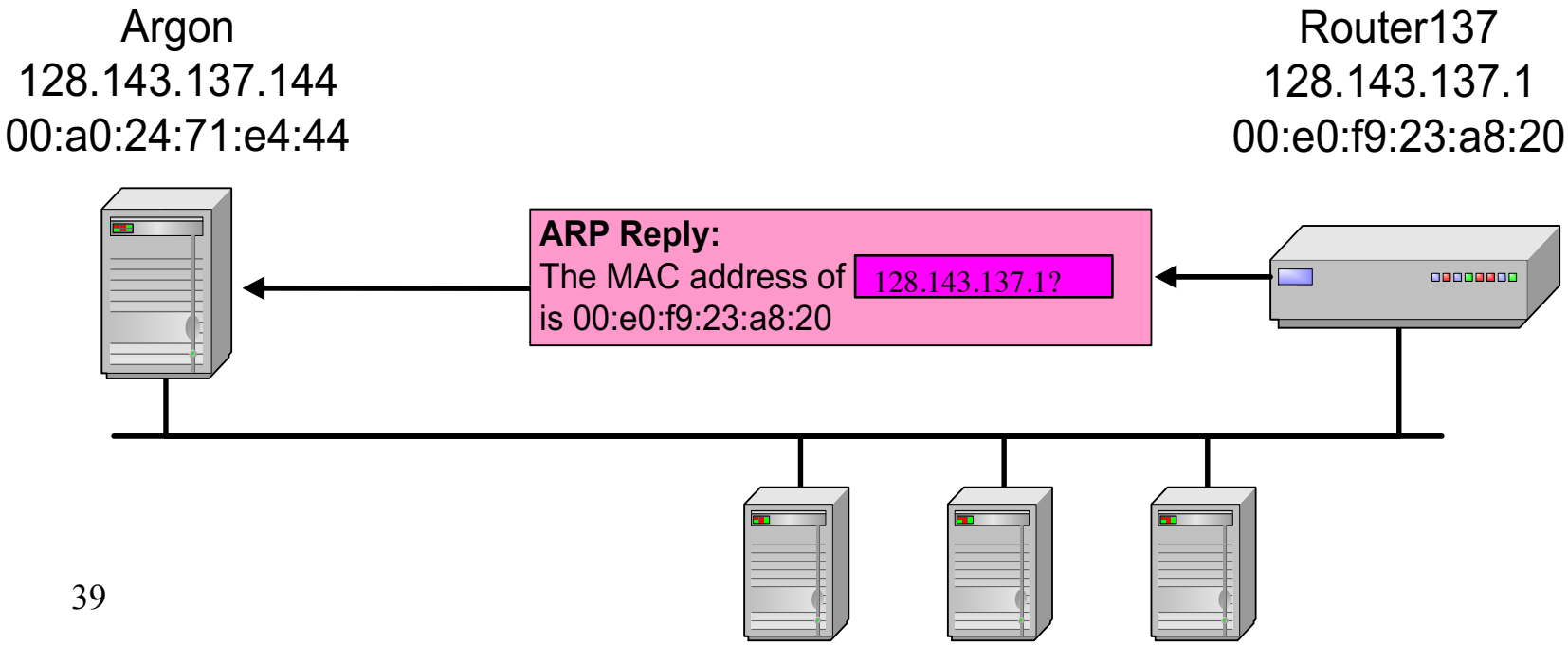
"What is the hardware address of 128.143.137.1?"



Address Translation with ARP

ARP Reply:

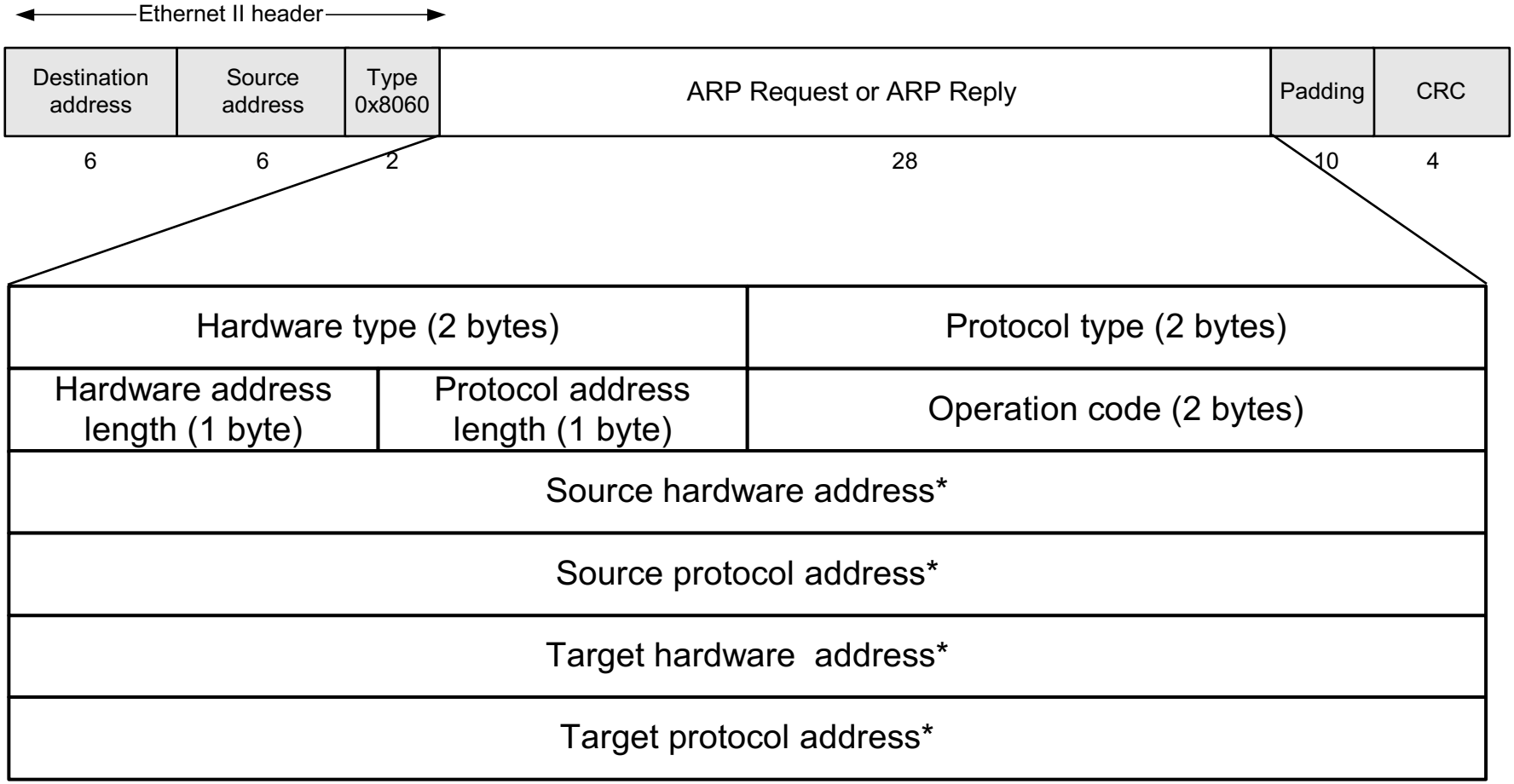
Router 137 responds with an ARP Reply which contains its hardware (MAC) address 00:e0:f9:23:a8:20 and is sent in data link frame to ARGON at address 00:a0:24:71:e4:44



ARP Transport

- ARP message travels in data portion of data link layer frame
- we say ARP message is *encapsulated in data link frame*
  - query/request has:
    - destination address: *ff:ff:ff:ff:ff:ff*
    - source address: *device's own MAC address*
  - reply has:
    - destination address: *of requesting device*
    - source address: *device's own MAC address*
- data portion padded with zeroes if ARP message is shorter than minimum data link frame (depends on hardware address length and network address length)
- for Ethernet: frame type field 0x0806 used for ARP

ARP Packet on Ethernet Link



* Note: The length of the address fields is determined by the corresponding address length fields

Example content of ARP packet

- *ARP Request from Argon:*

  Source hardware address: 00:a0:24:71:e4:44
  Source protocol address: 128.143.137.144
  Target hardware address: 00:00:00:00:00:00
  Target protocol address: 128.143.137.1

- *ARP Reply from Router137:*

  Source hardware address: 00:e0:f9:23:a8:20
  Source protocol address: 128.143.137.1
  Target hardware address: 00:a0:24:71:e4:44
  Target protocol address: 128.143.137.144

→ Result: an entry in ARP cache:

(128.143.137.1) at 00:e0:f9:23:a8:20 [ether] on eth0

ARP Packet Format

- general: can be used with
  - arbitrary hardware address (not just Ethernet)
  - arbitrary protocol address (not just IP)
- variable length address fields (depends on type of datalink protocol)

Retention of Bindings

- sending an ARP request/reply for each IP datagram to same IP address is inefficient
- solution -> maintain a table of bindings
  - devices maintain a cache of currently used IP addresses and their corresponding hardware addresses
- effect
  - use ARP for first datagram to a destination, place results in table, then quick lookup for MAC address (subsequent packets sent to that same IP address)

ARP Cache

- A caches (saves) B's IP-MAC address pair in an ARP table until information becomes old (times out)

  ARP table: < IP address; MAC address; TTL>

  - TTL (Time To Live): time after which address mapping will be forgotten (used to be 20 mins, now with mobile devices it is barely 30secs)
  - known as soft state: information that times out (goes away) unless refreshed
  - nodes sometimes will send a query to check if remote node is still "valid" - if a response is received, it renews the TTL
- B should cache A's information too, sometimes it needs a "data" request from B to A to enable cache
- ARP is "plug-and-play":
  - nodes create their ARP tables *without intervention from net administrator*

Example ARP Cache (table)

(128.143.71.37) at 00:10:4B:C5:D1:15 [ether] on eth0
(128.143.71.36) at 00:B0:D0:E1:17:D5 [ether] on eth0
(128.143.71.35) at 00:B0:D0:DE:70:E6 [ether] on eth0
(128.143.136.90) at 00:05:3C:06:27:35 [ether] on eth1
(128.143.71.34) at 00:B0:D0:E1:17:DB [ether] on eth0
(128.143.71.33) at 00:B0:D0:E1:17:DF [ether] on eth0

Things to know about ARP

- what happens if an ARP request is made for a non-existing host?
  - several ARP requests are made with increasing time intervals between requests.
  - eventually ARP gives up.
- what if a host sends an ARP request for its own IP address?
  - know as gratuitous ARP
  - no response is received hopefully……
  - this is useful for detecting if there is another device on the local network with the same Internet address (can happen by mistake)